# P7 — Handwritten Sudoku Reader and Solver

Search + CSP + Neural Network Student Project

**Project report**

Prepared from the supplied project source and verified pilot outputs

Submission status: implementation and demo verified; MNIST training and the full 100-puzzle experiment remain to be run on a network-enabled environment.

# 1. Problem and scope

The system reads a 9×9 Sudoku represented as 28×28 handwritten digit cells. A digit classifier predicts probabilities for digits 1–9. Duplicate clues expose likely recognition errors. A correction-search algorithm proposes low-cost edits, and a constraint-satisfaction solver checks whether each corrected grid can be solved. If no acceptable correction is found within the correction budget, the pipeline returns **unreadable** and lists low-confidence cells.

Scope: the project uses real isolated MNIST digit images to assemble synthetic Sudoku mosaics. It assumes the 9×9 grid and cell boundaries are already known. It is not a camera-based Sudoku sheet detector and does not perform perspective correction or automatic grid localization.

# 2. Dataset and generated data

MNIST supplies 28×28 grayscale digit images and labels. The data script downloads the official 60,000-image training and 10,000-image test partitions, makes a fixed-seed 50,000/10,000 split from the training partition, removes class 0, and scales pixels to float32 in [0,1]. The official test partition is held out from fitting and calibration. Sudoku solutions are generated by randomized backtracking; clues are removed only when uniqueness is preserved. Mosaics are assembled from MNIST glyphs. Injected hard cells blend a true-class glyph with a wrong-class distractor at alpha 0.45.

The detailed field-by-field dictionary is in *data/DATA_DICTIONARY.md*. The real source is MNIST; puzzle grids, mosaic layouts, injected misreads, and their metadata are generated.

# 3. Neural network

The CNN receives one grayscale 28×28 cell and outputs nine logits for digits 1–9. Softmax converts logits to class probabilities. A positive scalar temperature is fitted on validation logits by minimizing validation negative log-likelihood. The test partition is not used for fitting or calibration. The baseline is Logistic Regression on flattened pixels using a fixed-seed subset of up to 20,000 training samples. Planned metrics are accuracy, confusion matrix, expected calibration error (ECE), and negative log-likelihood (NLL).

**Measured CNN results are intentionally not reported:** MNIST download was attempted during preparation but failed because the environment could not resolve the dataset hosts. Therefore no trained CNN weights or CNN-vs-Logistic-Regression measurements are bundled. Run *data/make_data.py* and *nn/train.py* on a network-enabled machine before claiming classifier performance.

# 4. Search-based correction

A state is the current 81-cell reading. An action changes a cell involved in a duplicate conflict to one of its top-three alternative classifier classes. The edit cost is max(0, log(p(old)/p(new))), so it is never negative. UCS prioritizes accumulated cost g; Greedy Best-First prioritizes a conflict lower bound h; A* prioritizes g+h. The A* lower bound uses disjoint duplicate-conflict edges: at least one endpoint of each selected edge must change, so summing the cheapest endpoint-change cost is admissible. The demo correction budget defaults to three.

# 5. Constraint satisfaction

The CSP contains 81 variables. A given cell has a singleton domain; a blank cell has domain {1,…,9}. Cells in the same row, column, or 3×3 box must differ. Backtracking uses MRV with degree tie-breaking. Forward Checking removes a newly assigned digit from its peers. AC-3 enforces arc consistency on binary inequality constraints. The three modes are BT, BT+FC, and BT+FC+AC-3. Counters record assignments, backtracks, pruned values, and nodes. Optional uniqueness mode stops after two solutions and accepts only exactly one.

# 6. Integrated pipeline

The integration is NN probabilities → initial AC-3 consistency check → correction Search → CSP solvability (and optional uniqueness) → correction log. Each log item records row, column, old digit, new digit, and nonnegative cost. If no candidate succeeds within the correction budget, the system returns unreadable and ranks the least-confident nonblank cells.

## Verified demo run

Command: `python run_demo.py --method ucs --max-corrections 3`. The controlled synthetic demo corrected two conflicting readings (1→9 and 1→4), returned a solved grid, and reported 4 expanded / 27 generated Search states. These are synthetic probability results, not CNN inference.

# 7. Experiments and pilot results

The experiment runner writes raw observations to CSV, mean and population standard deviation to JSON, and plots. The included pilot has five puzzles per misread level (0–4) and five seeds for the CSP experiment. It is a development pilot, not the required final 100-puzzle-per-level evaluation.

| Exp. 1 · misreads | Method | Recovery mean ± SD | Expanded nodes mean |
|---|---|---|---|
| 2 | UCS | 1.00 ± 0.00 | 4.0 |
| 2 | GREEDY | 0.80 ± 0.40 | 340.4 |
| 2 | ASTAR | 1.00 ± 0.00 | 4.0 |

| Exp. 2 · givens | BT nodes | BT+FC nodes | BT+FC+AC-3 nodes |
|---|---|---|---|
| 25 | 267.2 | 272.4 | 26.2 |
| 30 | 82.4 | 80.6 | 6.0 |
| 35 | 49.6 | 48.4 | 1.2 |

| Exp. 3 · 2 misreads | Recovery mean ± SD |
|---|---|
| NN-guided synthetic probabilities | 1.00 ± 0.00 |
| No correction | 0.00 ± 0.00 |
| Random correction | 0.00 ± 0.00 |

Interpretation: in this small synthetic pilot, UCS and A* recovered all tested cases at two injected errors with four mean expanded nodes; Greedy had lower recovery and much higher search effort. AC-3 reduced CSP nodes substantially on these generated instances. These observations must not be generalized to real handwriting or treated as the full required evaluation.

# 8. Tests, reproducibility and limitations

The test suite contains 17 tests and passed with `pytest -q` in the preparation environment. It includes normal, edge, impossible, Search, and integration cases. The project uses pinned package requirements and fixed seeds for data generation and experiments. `pytest.ini` sets the repository root on the import path so both pytest invocation styles work.

## Known limitations and remaining finalization

• MNIST download and CNN training have not been run in this environment because DNS/network access to the dataset hosts failed.
• The full 100-puzzle-per-misread-level experiment has not been completed here; only the clearly labelled five-puzzle pilot is included.
• The mosaic path assumes known cell boundaries and is not a photographed-board reader.
• Team member contribution statements and authentic student git history must be supplied by the actual team.

# 9. Conclusion

The repository contains the intended modular data, NN, Search, CSP, integration, evaluation, and test code. The synthetic pipeline demo and tests are verified. The project should be considered implementation-complete at prototype level but not final-results-complete until MNIST training, CNN-vs-baseline metrics, and the full experiment runs are produced on a network-enabled environment. Update this report and the slide deck with those measured results before submission.

## References

Y. LeCun, C. Cortes, C. J. C. Burges, The MNIST Database of Handwritten Digits, http://yann.lecun.com/exdb/mnist/. The data is not redistributed in this archive.